

Notes on Ch 16 - Factors

Norman Lim

2025-07-30

Prerequisites

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.1.0
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Factor Basics

- Disadvantages of using strings to record variables:

1. Prone to typos.
2. It doesn't sort in a useful way.

Example:

```
# Intended data
x1 <- c("Dec", "Apr", "Jan", "Mar")

# Data with typo
x2 <- c("Dec", "Apr", "Jam", "Mar")

# sorting is not meaningful here
sort(x1)
```

```
## [1] "Apr" "Dec" "Jan" "Mar"
```

Solution:

- Convert the data into factor.

Steps:

1. Create valid levels.
2. Create a factor.

Demonstration:

```
# Step 1: create valid levels
```

```
month_levels = c(  
  "Jan", "Feb", "Mar", "Apr", "May", "Jun",  
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"  
)
```

```
# Step 2: create a factor
```

```
y1 <- factor(x1, levels = month_levels)  
y1
```

```
## [1] Dec Apr Jan Mar
```

```
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

```
sort(y1)
```

```
## [1] Jan Mar Apr Dec
```

```
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Disadvantage: - Any values not in the level will be silently converted to NA:

```
y2 <- factor(x2, levels = month_levels)  
y2
```

```
## [1] Dec Apr <NA> Mar
```

```
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Better way: - use `forcats::fct()` instead of `factor()` to convert strings into factors:

```
y2 <- fct(x2, levels = month_levels)
```

Q: What happens when we omit the levels? A: For `factor()`, default level will be assigned, in alphabetical order. For `fct()` the default level is sorted according to the order of appearance:

```
factor(x1)
```

```
## [1] Dec Apr Jan Mar
```

```
## Levels: Apr Dec Jan Mar
```

```
fct(x1)
```

```
## [1] Dec Apr Jan Mar
```

```
## Levels: Dec Apr Jan Mar
```

- Levels can be accessed using `levels()`:

```
levels(y2)
```

```
## [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

- When reading data with **readr**, we can create a factor on the spot using `col_factor()`:

```
csv <- "  
month,value  
Jan,12  
Feb,56  
Mar,12"  
  
df <- read_csv(csv, col_types = cols(month = col_factor(month_levels)))  
df$month
```

```
## [1] Jan Feb Mar
```

```
## Levels: Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
```

Demo with factors using GSS data

```
gss_cat
```

```
## # A tibble: 21,483 x 9
##   year marital      age race  rincome      partyid      relig denom tvhours
##   <int> <fct>      <int> <fct> <fct>      <fct>      <fct> <fct>      <int>
## 1  2000 Never married    26 White $8000 to 9999 Ind,near ~ Prot~ Sout~      12
## 2  2000 Divorced        48 White $8000 to 9999 Not str r~ Prot~ Bapt~      NA
## 3  2000 Widowed         67 White Not applicable Independe~ Prot~ No d~       2
## 4  2000 Never married    39 White Not applicable Ind,near ~ Orth~ Not ~       4
## 5  2000 Divorced         25 White Not applicable Not str d~ None  Not ~       1
## 6  2000 Married         25 White $20000 - 24999 Strong de~ Prot~ Sout~      NA
## 7  2000 Never married    36 White $25000 or more Not str r~ Chri~ Not ~       3
## 8  2000 Divorced         44 White $7000 to 7999 Ind,near ~ Prot~ Luth~      NA
## 9  2000 Married         44 White $25000 or more Not str d~ Prot~ Other       0
## 10 2000 Married         47 White $25000 or more Strong re~ Prot~ Sout~       3
## # i 21,473 more rows
```

- Viewing the levels using `count()`:

```
gss_cat |>
  count(race)
```

```
## # A tibble: 3 x 2
##   race      n
##   <fct> <int>
## 1 Other  1959
## 2 Black 3129
## 3 White 16395
```

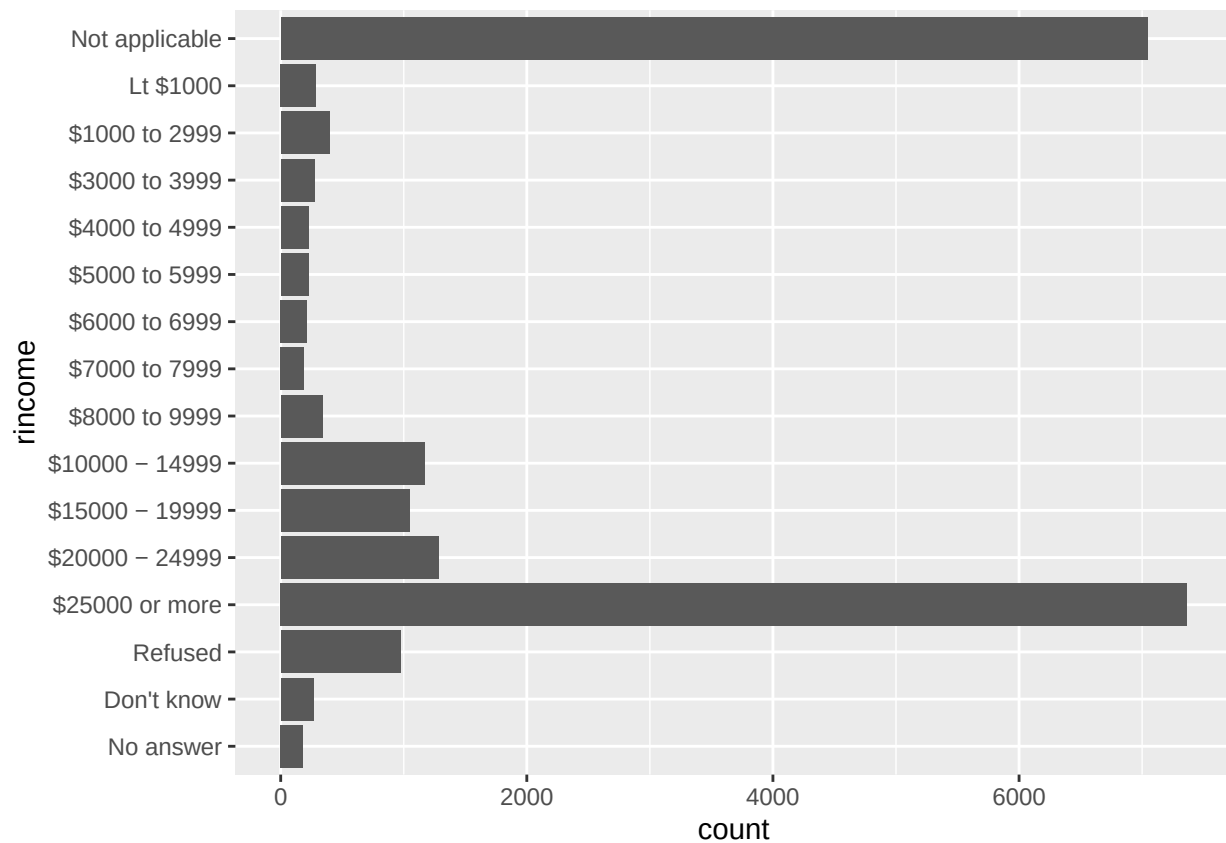
Two most important operations when working with factors: 1. Changing the order of the levels.
2. Changing the values of the levels.

Exercises

1. Explore the distribution of `rincome` (reported income). What makes the default bar chart hard to understand? How could you improve the plot.

Ans.

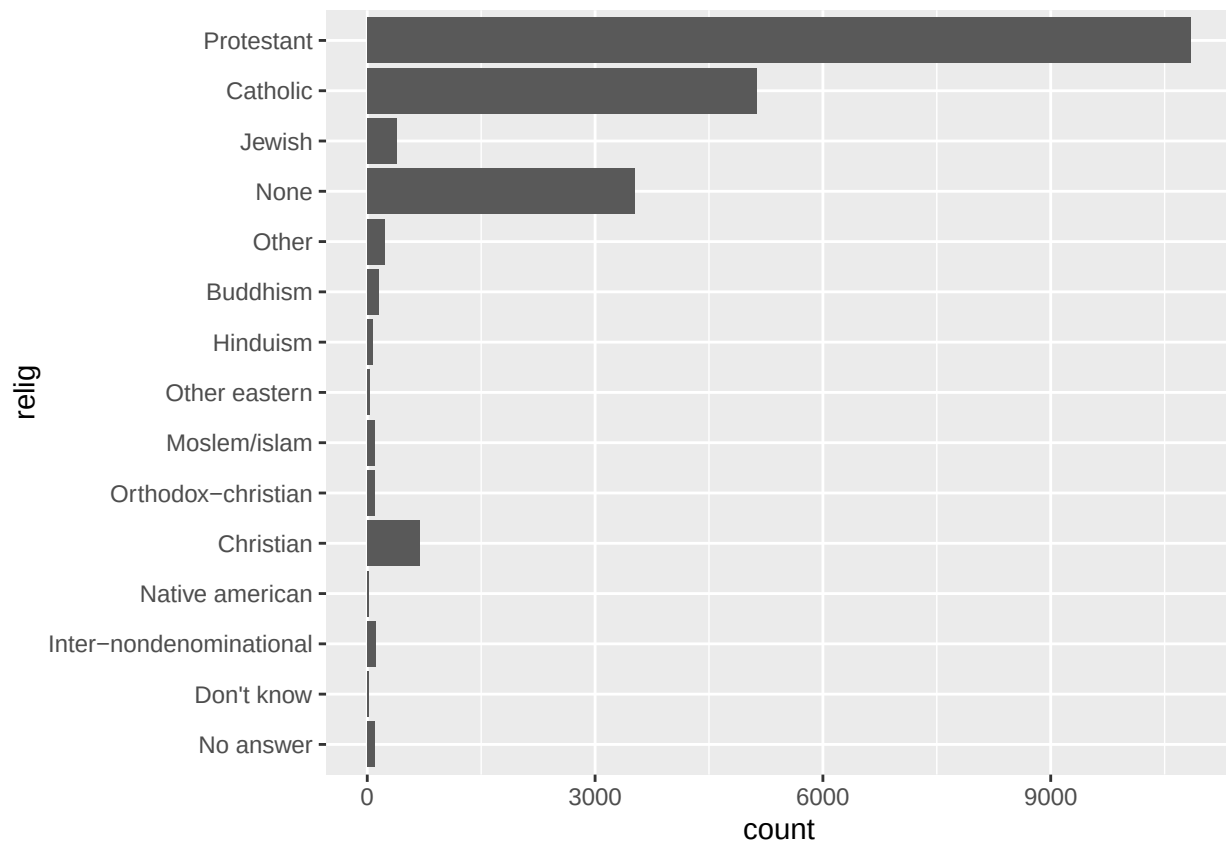
```
gss_cat |>
  ggplot(aes(x = rincome)) +
  geom_bar() +
  coord_flip()
```



The income were given terms of salary ranges and the labels are long. Flipping the coordinates or using a horizontal bar char would make it easier to understand. Ordering the bars will also make it easier to understand.

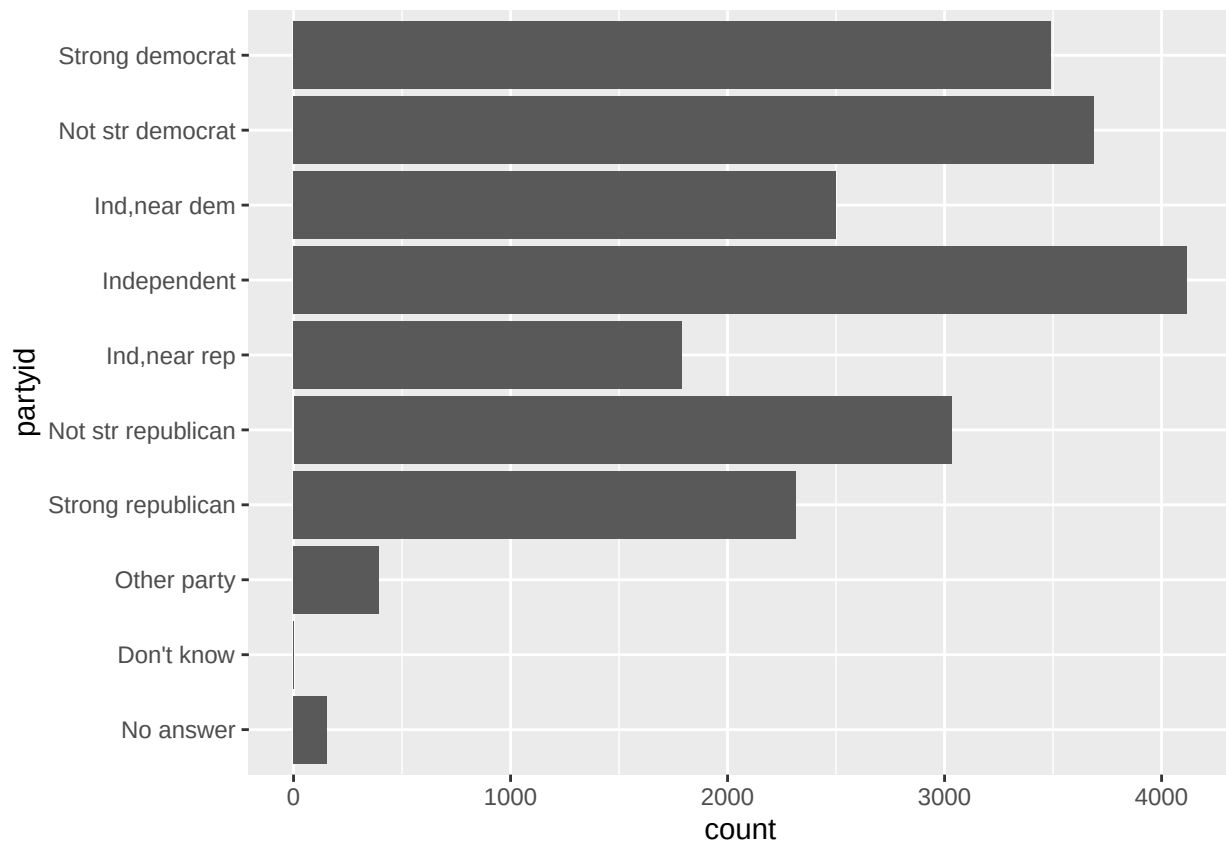
2. What is the most common **relig** in this survey? What is the most common **partyid**?

```
gss_cat |>
  ggplot(aes(x = relig)) +
  geom_bar() +
  coord_flip()
```



Protestant is the most common religion for the respondents.

```
gss_cat |>
  ggplot(aes(x = partyid)) +
  geom_bar() +
  coord_flip()
```



The most common partyid is 'Independent'.

3. Which relig does denom (denomination) apply to? How can you find out with a table? How can you find out with a visualization?

table:

```
table3 <- gss_cat |>
  select(relig, denom) |>
  group_by(relig, denom) |>
  summarize(
    count = n()
  ) |>
  ungroup()
```

`summarise()` has grouped output by 'relig'. You can override using the
`.groups` argument.

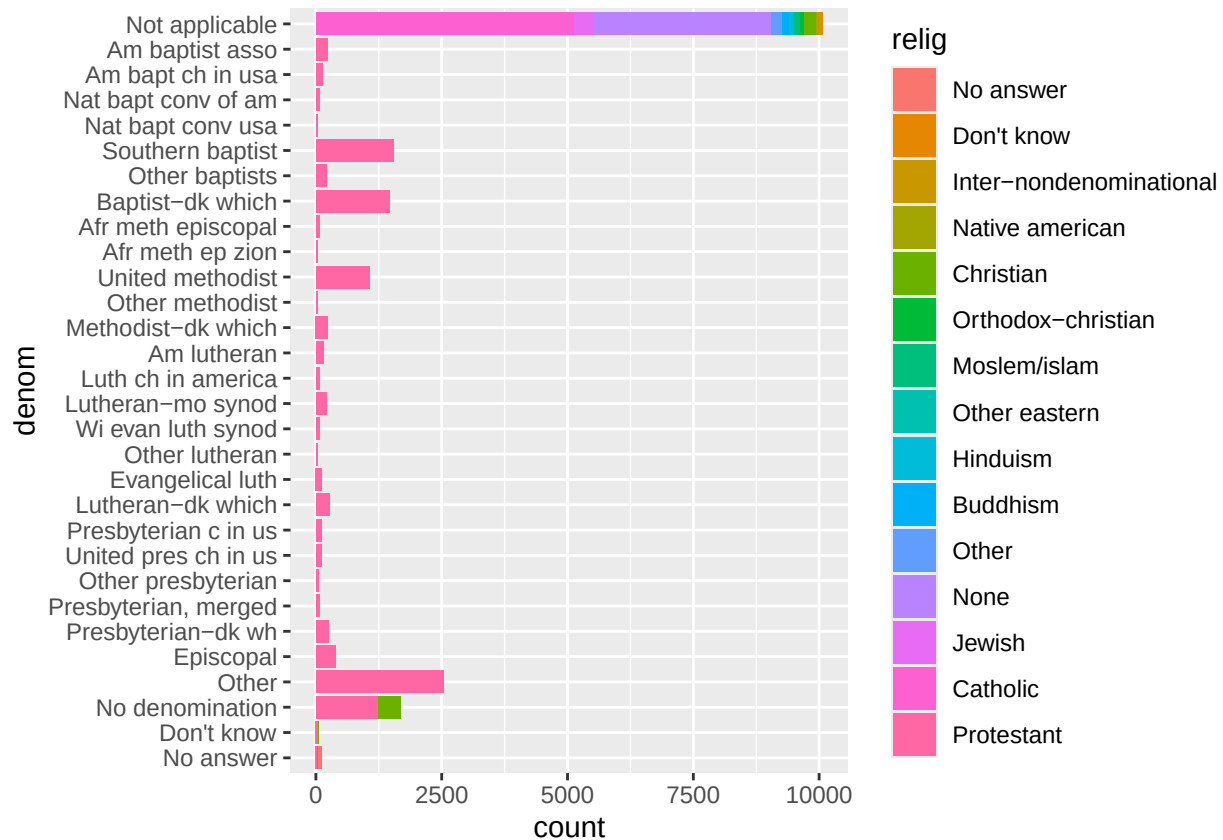
table3

```
## # A tibble: 47 x 3
##   relig          denom      count
##   <fct>         <fct>    <int>
## 1 No answer      No answer      93
## 2 Don't know     Not applicable  15
## 3 Inter-nondenom Not applicable 109
## 4 Native american Not applicable  23
## 5 Christian      No answer       2
## 6 Christian      Don't know     11
## 7 Christian      No denomination 452
```

```
## 8 Christian                Not applicable    224
## 9 Orthodox-christian      Not applicable    95
## 10 Moslem/islam           Not applicable   104
## # i 37 more rows
```

data viz:

```
table3 |> ggplot(aes(x = denom, y = count, fill = relig)) +
  geom_col() +
  coord_flip()
```



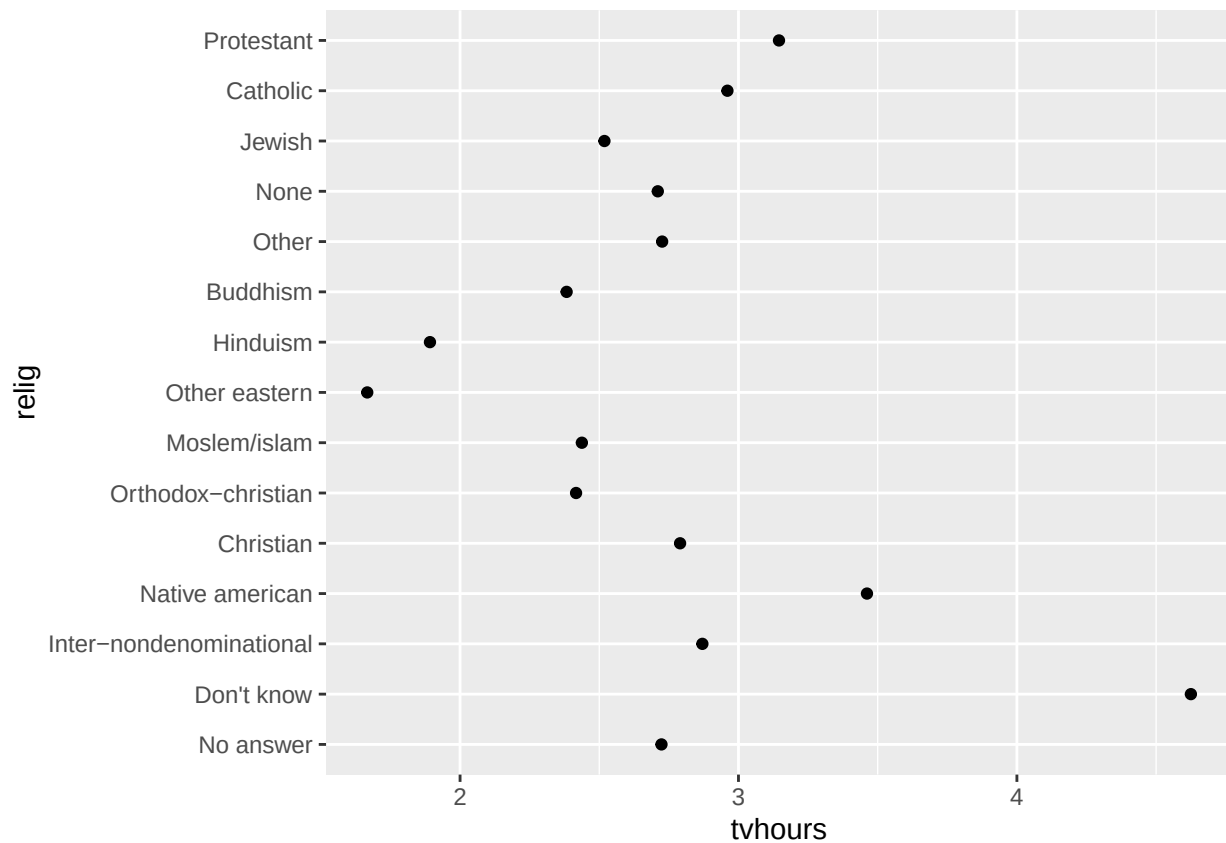
denom refers to the religious sect(?).

Modifying factor order

Example showing a plot with no sorting order applied:

```
relig_summary <- gss_cat |>
  group_by(relig) |>
  summarize(
    tvhours = mean(tvhours, na.rm = TRUE),
    n = n()
  )

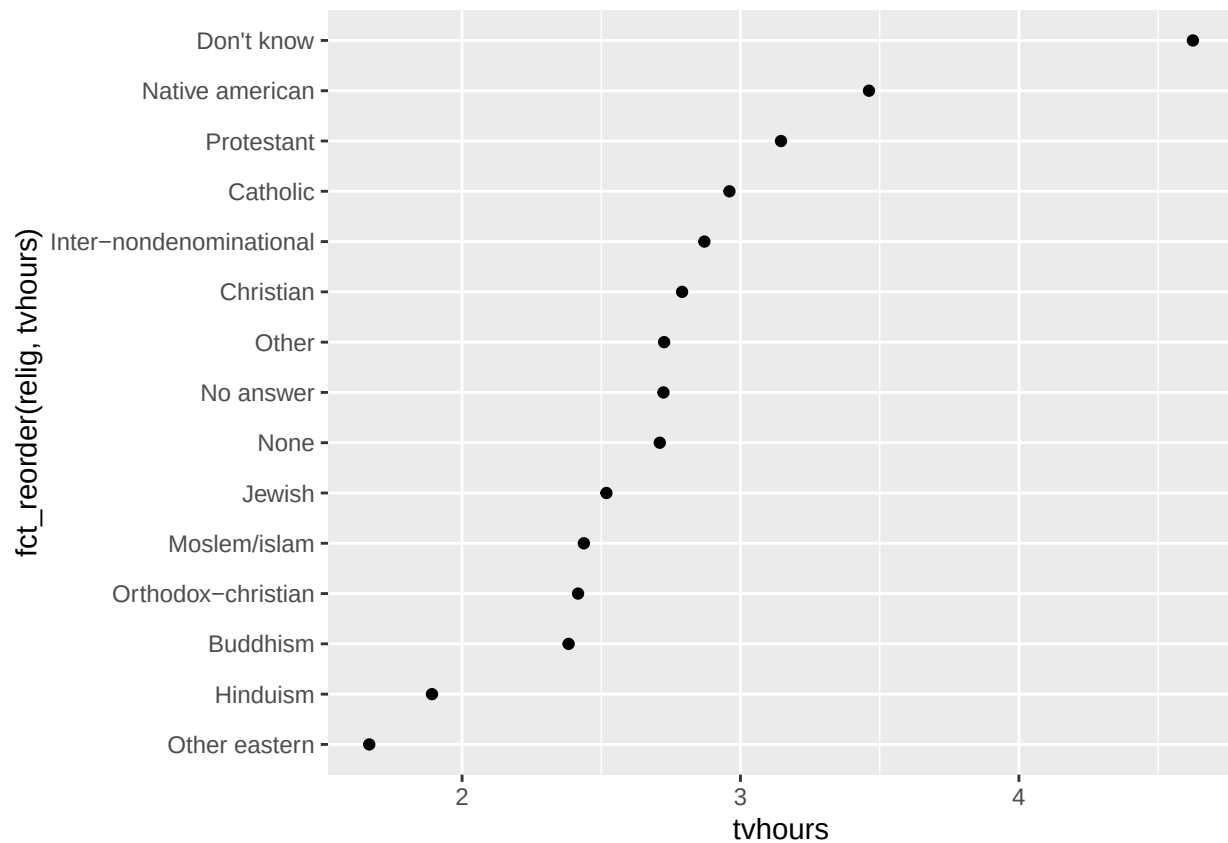
ggplot(relig_summary, aes(x = tvhours, y = relig)) +
  geom_point()
```



The plot is hard to read because there is no overall pattern.

Plot ordered using `fct_reorder()`:

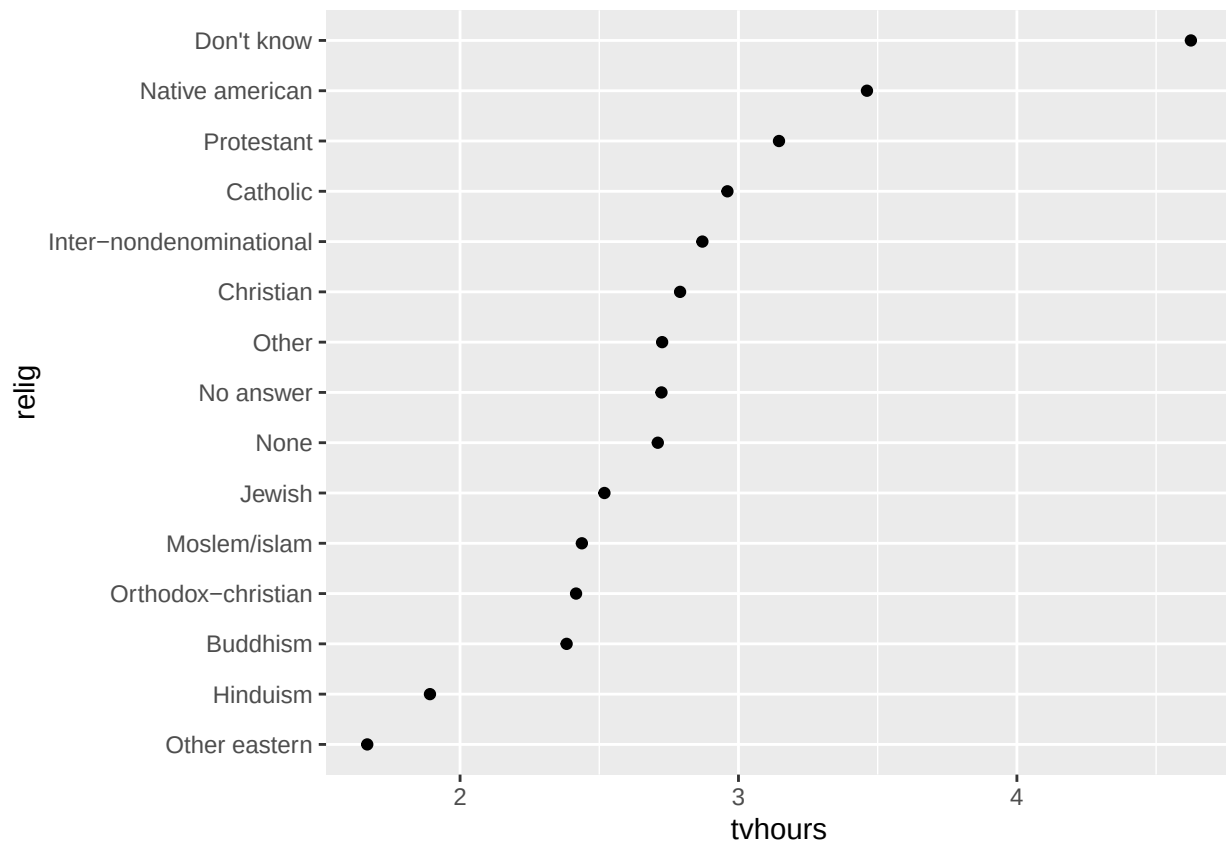
```
relig_summary |>
  ggplot(aes(
    x = tvhours,
    y = fct_reorder(relig, tvhours)
  )) +
  geom_point()
```

Here `relig` is reordered according to the number of `tvhours`.

- When the transformations are more complicated, it is better to do that inside the `mutate()` function rather than 'on-the-fly'.

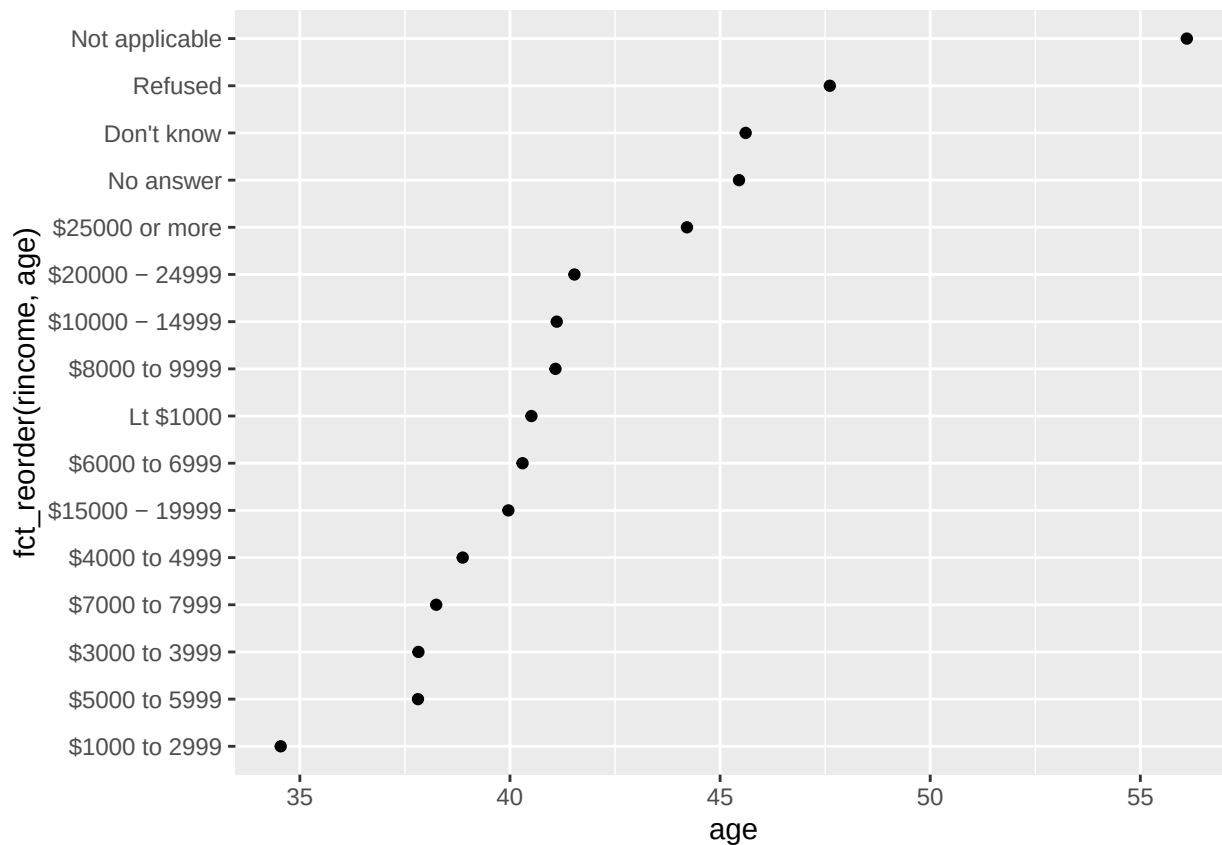
```
relig_summary |>
  mutate(
    relig = fct_reorder(relig, tvhours)
  ) |>
  ggplot(aes(x = tvhours, y = relig)) +
  geom_point()
```



Plot exploring how the average age varies across reported income levels. Again, `fct_reorder()` was used here.

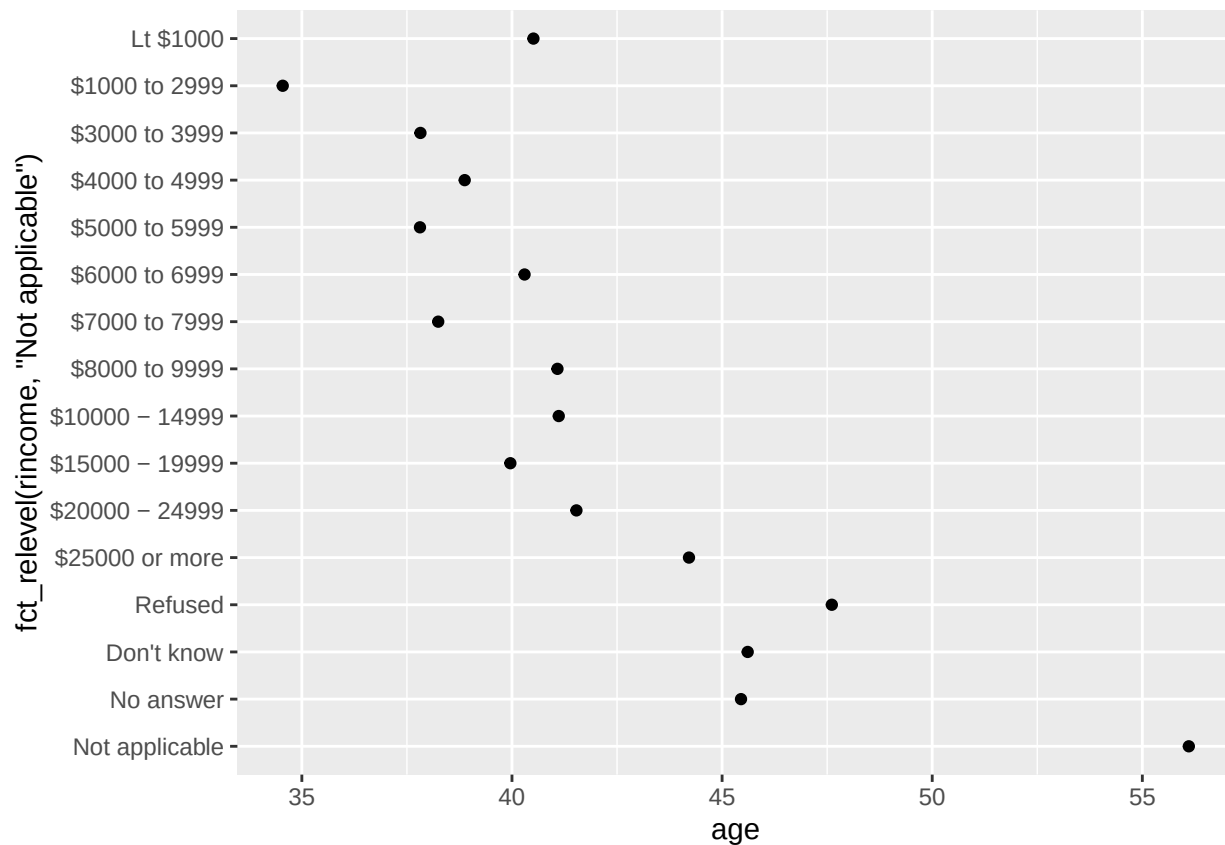
```
rincome_summary <- gss_cat |>
  group_by(rincome) |>
  summarize(
    age = mean(age, na.rm = TRUE),
    n = n()
  )

rincome_summary |>
  ggplot(aes(
    x = age,
    y = fct_reorder(rincome, age)
  )) +
  geom_point()
```



- This time, the technique or reordering does not work well. This is because `rincome` already has a principled order.
- We should reserve `fct_reorder` for factors whose levels are arbitrarily ordered.
- If there is a sensible level already, and we only want to tweak it a bit, `fct_relevel()` is useful.
- Example where `fct_relevel()` is used to move “Not applicable” to the bottom of the plot:

```
rincome_summary |> ggplot(aes(
  x = age,
  y = fct_relevel(rincome, "Not applicable")
)) +
  geom_point()
```

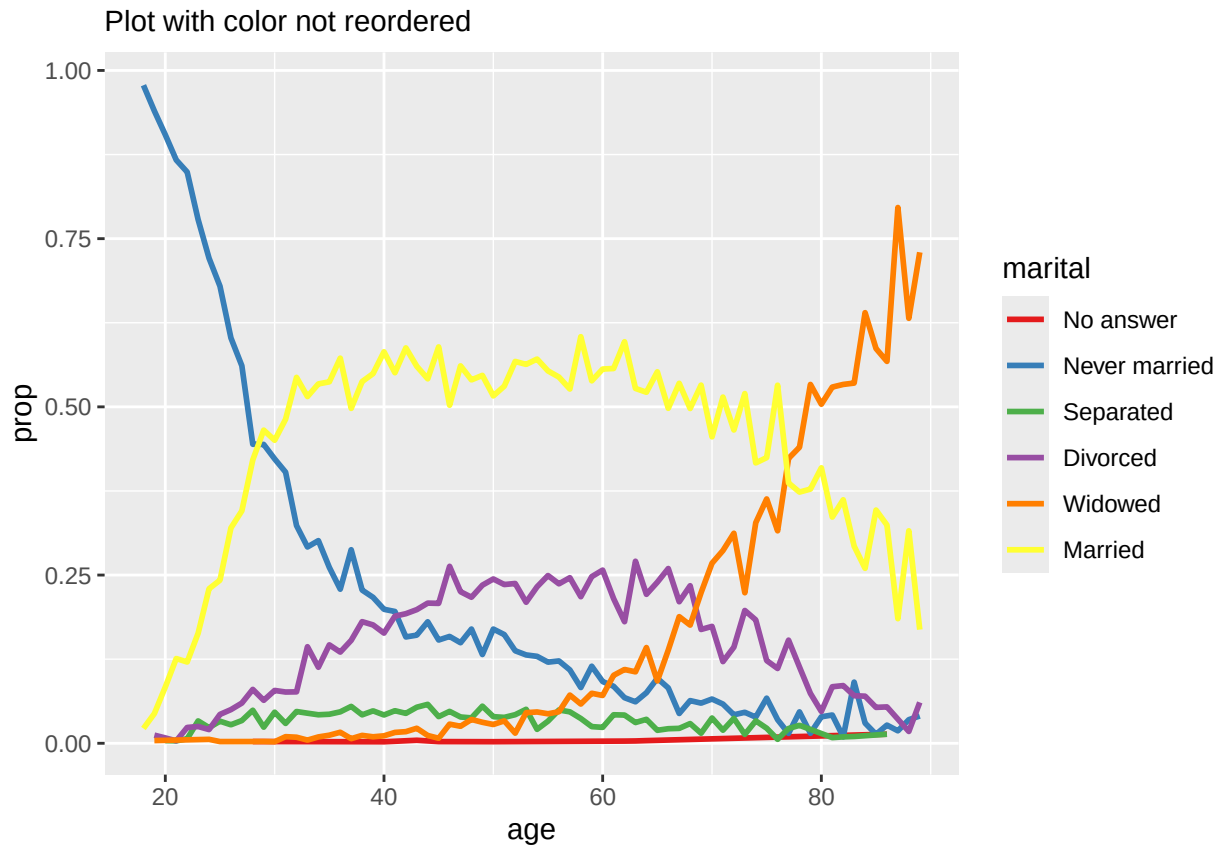


It looks like most churchgoers in the survey are senior citizens who have already retired (no income).

- Example of using `fct_reorder()` to improve coloring on the plot:

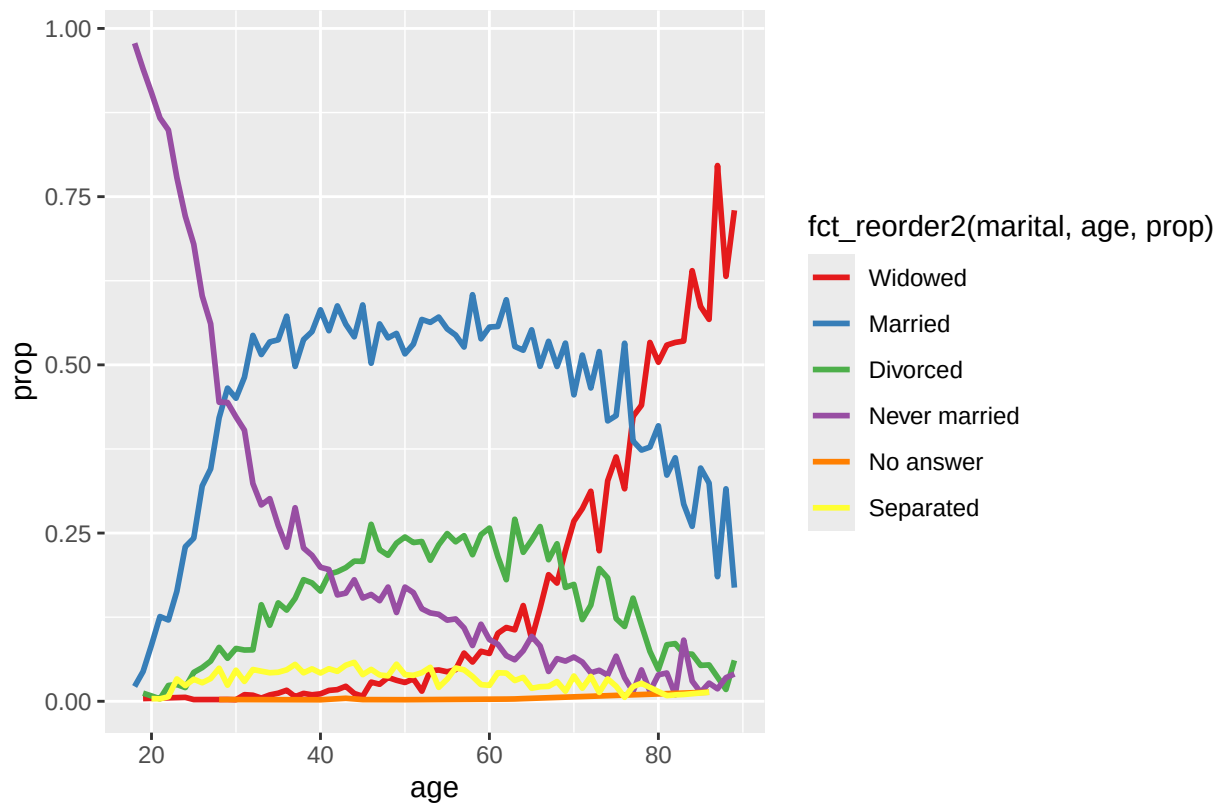
```
by_age <- gss_cat |>
  filter(!is.na(age)) |>
  count(age, marital) |>
  group_by(age) |>
  mutate(
    prop = n / sum(n)
  )

by_age |> ggplot(aes(
  x = age,
  y = prop,
  color = marital
)) +
  geom_line(linewidth = 1) +
  scale_color_brewer(palette = "Set1") +
  labs(
    subtitle = "Plot with color not reordered"
  )
```



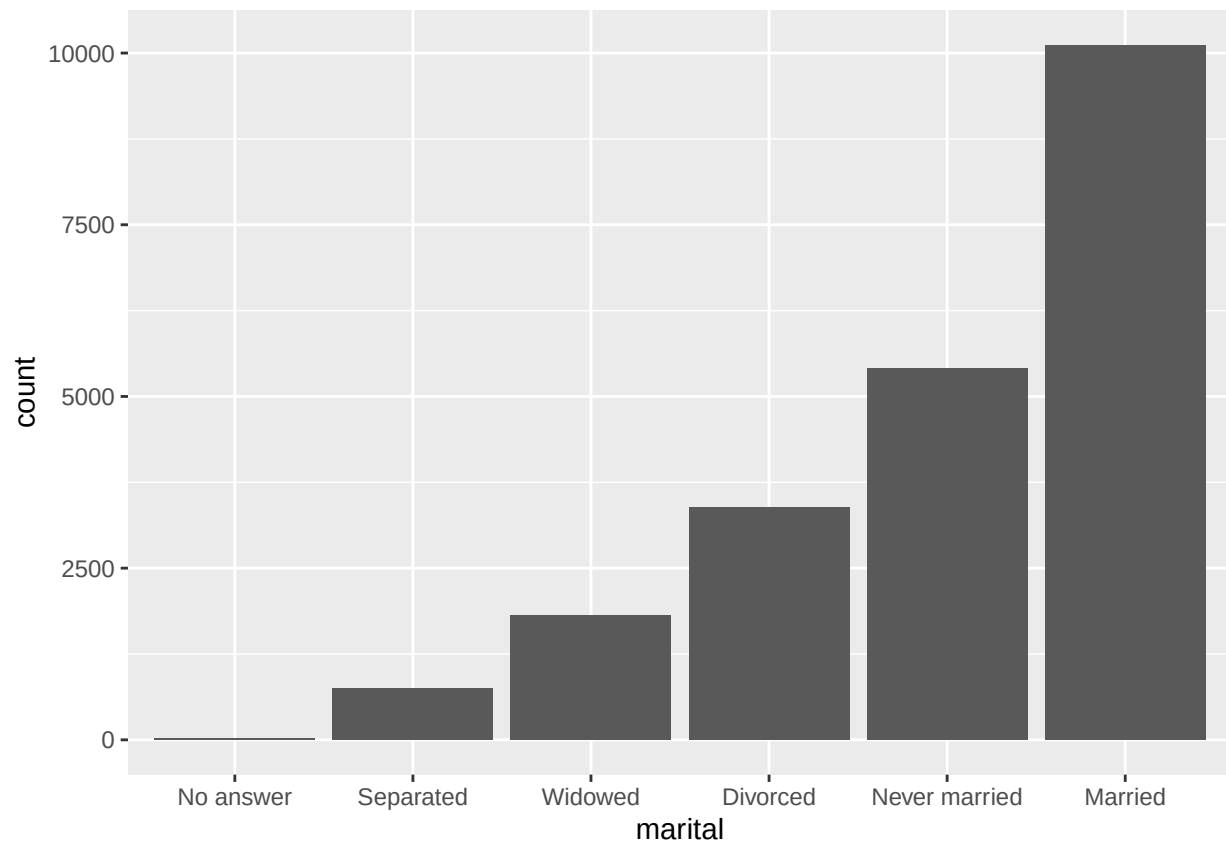
```
by_age |> ggplot(aes(
  x = age,
  y = prop,
  color = fct_reorder2(marital, age, prop)
)) +
  geom_line(linewidth = 1) +
  scale_color_brewer(palette = "Set1") +
  labs(
    subtitle = "Same plot but with color reordered"
  )
```

Same plot but with color reordered



- For bar plots, `fct_infreq()` can be used to order levels in decreasing frequency.
- `fct_infreq()` can be paired with `fct_rev()` if you want bar plot rendered in increasing frequency.

```
gss_cat |>
  mutate(
    marital = marital |> fct_infreq() |> fct_rev()
  ) |>
  ggplot(aes(x = marital)) +
  geom_bar()
```



Exercises

1. There are some suspiciously high numbers in `tvhours`. Is the mean a good summary()?

Ans.

```
relig_summary |>
  arrange(desc(n))
```

```
## # A tibble: 15 x 3
##   relig          tvhours      n
##   <fct>          <dbl> <int>
## 1 Protestant      3.15 10846
## 2 Catholic        2.96  5124
## 3 None            2.71  3523
## 4 Christian       2.79   689
## 5 Jewish          2.52   388
## 6 Other           2.73   224
## 7 Buddhism        2.38   147
## 8 Inter-nondenom  2.87   109
## 9 Moslem/islam    2.44   104
## 10 Orthodox-christian 2.42    95
## 11 No answer       2.72    93
## 12 Hinduism        1.89    71
## 13 Other eastern   1.67    32
## 14 Native american  3.46    23
## 15 Don't know      4.62    15
```

No. Averaging `tvhours` for each `relig` will not give us a fair result since the number of respondents for

each `relig` is so unbalanced – more than 10,000 identified as protestant but only 71 identified as Hindu, for example. Also, there might be other factors at play here, such as economic status and hours spent watching tv.

2. For each factor in `gss_cat`, identify whether the order of the levels is arbitrary or principled.

Ans.

```
levels(gss_cat$marital)
```

```
## [1] "No answer"      "Never married" "Separated"      "Divorced"
## [5] "Widowed"        "Married"
```

```
levels(gss_cat$race)
```

```
## [1] "Other"          "Black"          "White"          "Not applicable"
```

```
levels(gss_cat$rincome)
```

```
## [1] "No answer"      "Don't know"      "Refused"          "$25000 or more"
## [5] "$20000 - 24999" "$15000 - 19999" "$10000 - 14999" "$8000 to 9999"
## [9] "$7000 to 7999" "$6000 to 6999" "$5000 to 5999" "$4000 to 4999"
## [13] "$3000 to 3999" "$1000 to 2999" "Lt $1000"         "Not applicable"
```

```
levels(gss_cat$partyid)
```

```
## [1] "No answer"      "Don't know"      "Other party"
## [4] "Strong republican" "Not str republican" "Ind,near rep"
## [7] "Independent"      "Ind,near dem"      "Not str democrat"
## [10] "Strong democrat"
```

```
levels(gss_cat$relig)
```

```
## [1] "No answer"      "Don't know"
## [3] "Inter-nondenominational" "Native american"
## [5] "Christian"      "Orthodox-christian"
## [7] "Moslem/islam"    "Other eastern"
## [9] "Hinduism"        "Buddhism"
## [11] "Other"           "None"
## [13] "Jewish"          "Catholic"
## [15] "Protestant"      "Not applicable"
```

```
levels(gss_cat$denom)
```

```
## [1] "No answer"      "Don't know"      "No denomination"
## [4] "Other"          "Episcopal"       "Presbyterian-dk wh"
## [7] "Presbyterian, merged" "Other presbyterian" "United pres ch in us"
## [10] "Presbyterian c in us" "Lutheran-dk which" "Evangelical luth"
## [13] "Other lutheran"    "Wi evan luth synod" "Lutheran-mo synod"
## [16] "Luth ch in america" "Am lutheran"      "Methodist-dk which"
## [19] "Other methodist"   "United methodist" "Afr meth ep zion"
## [22] "Afr meth episcopal" "Baptist-dk which" "Other baptists"
## [25] "Southern baptist"  "Nat bapt conv usa" "Nat bapt conv of am"
## [28] "Am bapt ch in usa" "Am baptist asso"  "Not applicable"
```

marital: arbitrary race: arbitrary rincome: principled partyid: arbitrary relig: arbitrary denom: arbitrary

3. Why did moving “Not Applicable” to the front of the levels move it to the bottom of the plot()

Ans. Horizontal bar charts are rendered in reverse order. This behavior can also be seen in Python, where a table shown in decreasing order would be rendered in increasing order in a bar plot.

Modifying factor levels

- The most powerful tool for modifying factors is `fct_recode()`.
- `fct_recode()` allows us to recode, or change the value of each level.

Example:

```
gss_cat |> count(partyid)
```

```
## # A tibble: 10 x 2
##   partyid          n
##   <fct>          <int>
## 1 No answer      154
## 2 Don't know      1
## 3 Other party    393
## 4 Strong republican 2314
## 5 Not str republican 3032
## 6 Ind,near rep    1791
## 7 Independent    4119
## 8 Ind,near dem    2499
## 9 Not str democrat 3690
## 10 Strong democrat 3490
```

The default levels here are inconsistent.

- Tweaking the levels using `fct_recode()`:

```
gss_cat |>
  mutate(
    partyid = fct_recode(partyid,
      "Republican, strong" = "Strong republican",
      "Republican, weak" = "Not str republican",
      "Independent, near rep" = "Ind,near rep",
      "Independent near dem" = "Ind,near dem",
      "Democrat, weak" = "Not str democrat",
      "Democrat, strong" = "Strong democrat"
    )
  ) |>
  count(partyid)
```

```
## # A tibble: 10 x 2
##   partyid          n
##   <fct>          <int>
## 1 No answer      154
## 2 Don't know      1
## 3 Other party    393
## 4 Republican, strong 2314
## 5 Republican, weak 3032
## 6 Independent, near rep 1791
## 7 Independent    4119
## 8 Independent near dem 2499
## 9 Democrat, weak 3690
## 10 Democrat, strong 3490
```

- `fct_collapse()` is a useful tool for reducing the number of levels or lumping together some levels:

```
gss_cat |>
  mutate(
    partyid = fct_collapse(partyid,
      "other" = c("No answer", "Don't know", "Other party"),
      "rep"   = c("Strong republican", "Not str republican"),
      "ind"   = c("Ind,near rep", "Independent", "Ind,near dem"),
      "dem"   = c("Not str democrat", "Strong democrat")
    )
  ) |>
  count(partyid)
```

```
## # A tibble: 4 x 2
##   partyid     n
##   <fct>   <int>
## 1 other     548
## 2 rep     5346
## 3 ind     8409
## 4 dem     7180
```

Example of manually lumping to “Other” some categories:

```
gss_cat |>
  mutate(
    partyid = fct_recode(partyid,
      "Republican, strong" = "Strong republican",
      "Republican, weak"  = "Not str republican",
      "Independent, near rep" = "Ind,near rep",
      "Independent, near dem" = "Ind,near dem",
      "Democrat, weak"      = "Not str democrat",
      "Democrat, strong"   = "Strong democrat",
      "Other"              = "No answer",
      "Other"              = "Don't know",
      "Other"              = "Other party"
    )
  ) |>
  count(partyid)
```

```
## # A tibble: 8 x 2
##   partyid           n
##   <fct>         <int>
## 1 Other           548
## 2 Republican, strong 2314
## 3 Republican, weak  3032
## 4 Independent, near rep 1791
## 5 Independent       4119
## 6 Independent, near dem 2499
## 7 Democrat, weak    3690
## 8 Democrat, strong   3490
```

- Example using `fct_lump_*()` family of functions to lump small groups to “Other”:

```
gss_cat |>
  mutate(
    relig = fct_lump_lowfreq(relig)
  ) |>
  count(relig)
```

```
## # A tibble: 2 x 2
##   relig      n
##   <fct>    <int>
## 1 Protestant 10846
## 2 Other      10637
```

- To control how the small categories are lumped together, we can also use `fct_lump_n()` to tell dplyr how many categories to retain:

```
gss_cat |>
  mutate(
    relig = fct_lump_n(relig, n = 10)
  ) |>
  count(relig, sort = TRUE)
```

```
## # A tibble: 10 x 2
##   relig      n
##   <fct>    <int>
## 1 Protestant 10846
## 2 Catholic   5124
## 3 None       3523
## 4 Christian   689
## 5 Other       458
## 6 Jewish      388
## 7 Buddhism    147
## 8 Inter-nondenominational 109
## 9 Moslem/islam 104
## 10 Orthodox-christian 95
```

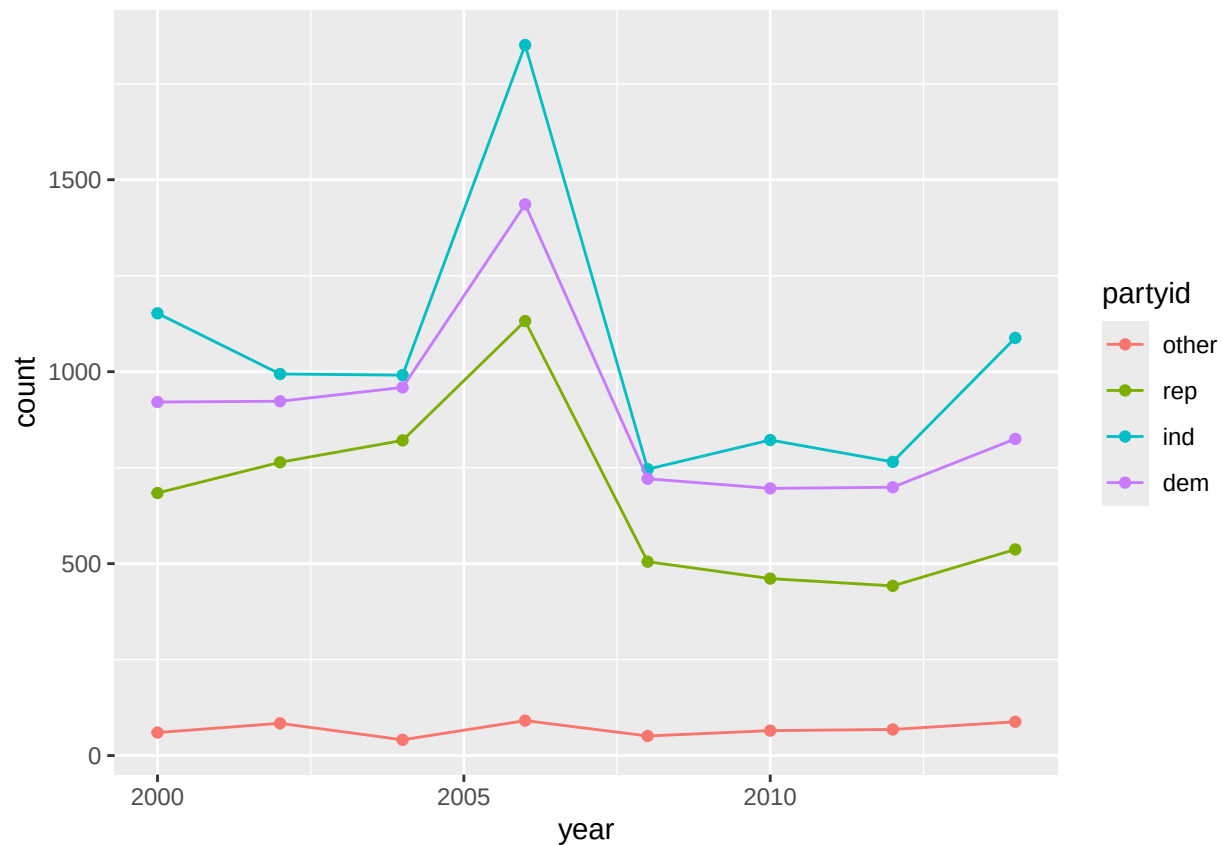
Exercises

1. How have the proportions of people identifying as Democrat, Republican, and Independent changed over time?

Ans.

```
gss_cat |>
  mutate(
    partyid = fct_collapse(partyid,
      "other" = c("No answer", "Don't know", "Other party"),
      "rep"   = c("Strong republican", "Not str republican"),
      "ind"   = c("Ind,near rep", "Independent", "Ind,near dem"),
      "dem"   = c("Not str democrat", "Strong democrat")
    )
  ) |>
  group_by(year, partyid) |>
  summarize(
    count = n()
  ) |>
  ggplot(aes(x = year, y = count, color = partyid)) +
  geom_point() +
  geom_line()
```

```
## `summarise()` has grouped output by 'year'. You can override using the
## `.groups` argument.
```



2. How could you collapse `rincome` into a small set of categories?

Ans. I would categorize income as low, medium, high, and others using `fct_recode()`

3. Notice there are 9 groups (excluding other) in the `fct_lump` example above. Why not 10?

Ans. Because the default `other_level` for `fct_lump` is considered as a level too.

Ordered factors

- A ranking can be created for factors using the `ordered()` function:

```
ordered(c("a", "b", "c"))
```

```
## [1] a b c
```

```
## Levels: a < b < c
```

- Ordered factors are particularly useful when mapping colors.